DO NOW — RECALL

Unfamiliar algorithm design and evaluation

Name: _____ Class: _____ Date: _____

1. What is a precondition?

2. Why write edge-case tests?

3. What does a loop invariant express?

4. C# array [4,1,4,2]: compute sum at indexes 0 and 2, and number of distinct values.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Requirement for an algorithm to work correctly.

2. Model answer

To expose faults on boundaries/unusual inputs.

3. Model answer

A property that stays true after each loop iteration.

4. Model answer

Indexes 0 and 2 sum8; distinct values {4,1,2} count3.

By the end, you can…

• State the precondition

• Trace the invariant

• Evaluate and test

Vocabulary — know these terms

invariant — Condition maintained throughout a loop.

precondition — Assumption required for correctness.

boundary test — Tests around value/size limits.

time complexity — Growth in work as input increases.

NEW LEARNING

Read, annotate and explain

State the precondition

The input must be sorted; equal values then appear consecutively.

Trace the invariant

Start with one distinct value for a nonempty array. Each change from previous value adds one.

Evaluate and test

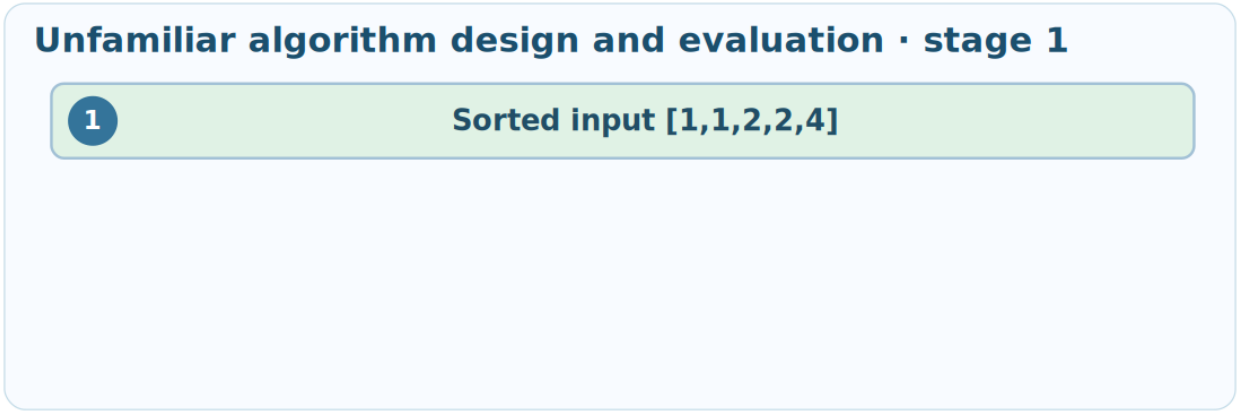
The algorithm scans once O(n) time, O(1) extra space; test empty, all equal and all distinct inputs.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

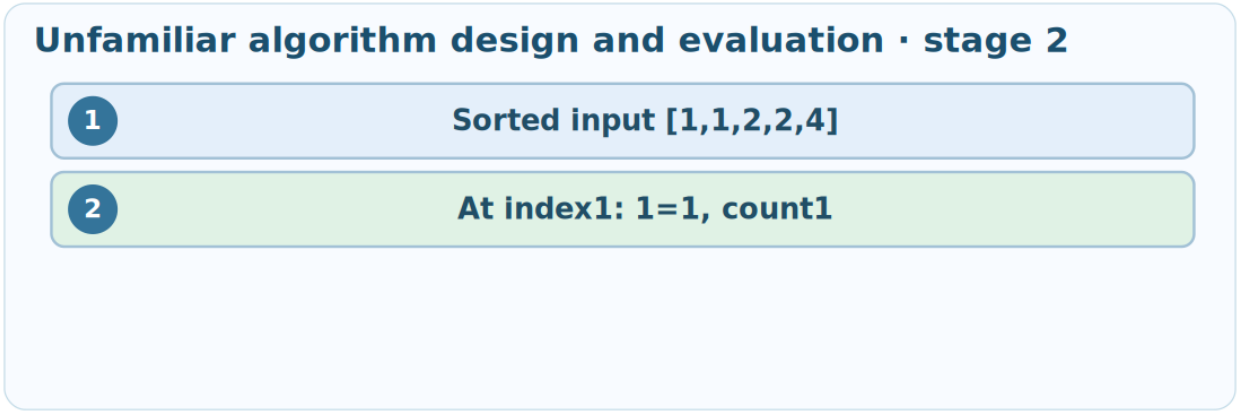
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Unfamiliar algorithm design and evaluation · stage 1; Sorted input [1,1,2,2,4]

New Learning

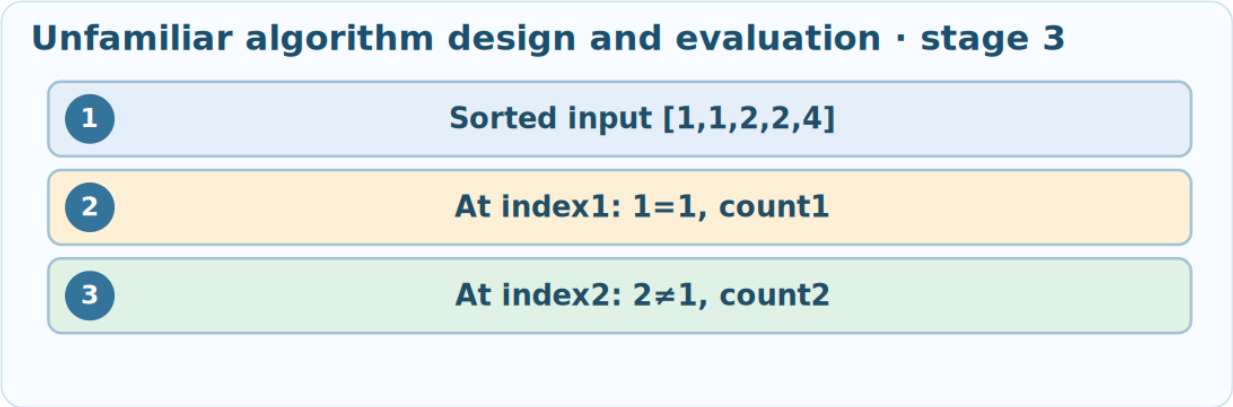


Unfamiliar algorithm design and evaluation · stage 2; Sorted input [1,1,2,2,4]; At index1: 1=1, count1

New Learning · original lesson diagrams

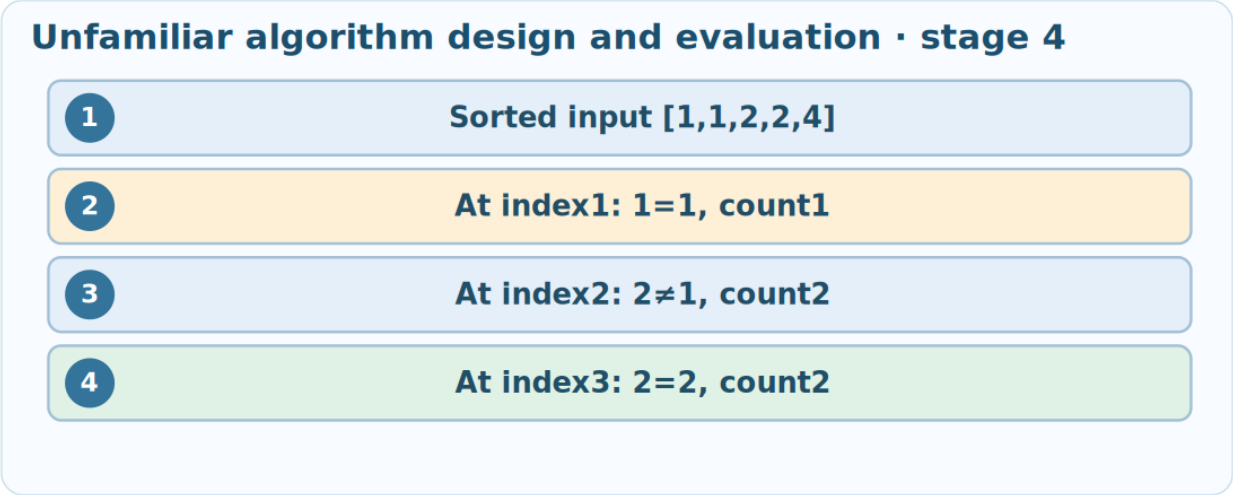
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Unfamiliar algorithm design and evaluation · stage 3; Sorted input [1,1,2,2,4]; At index1: 1=1, count1; At index2: 2≠1, count2

New Learning

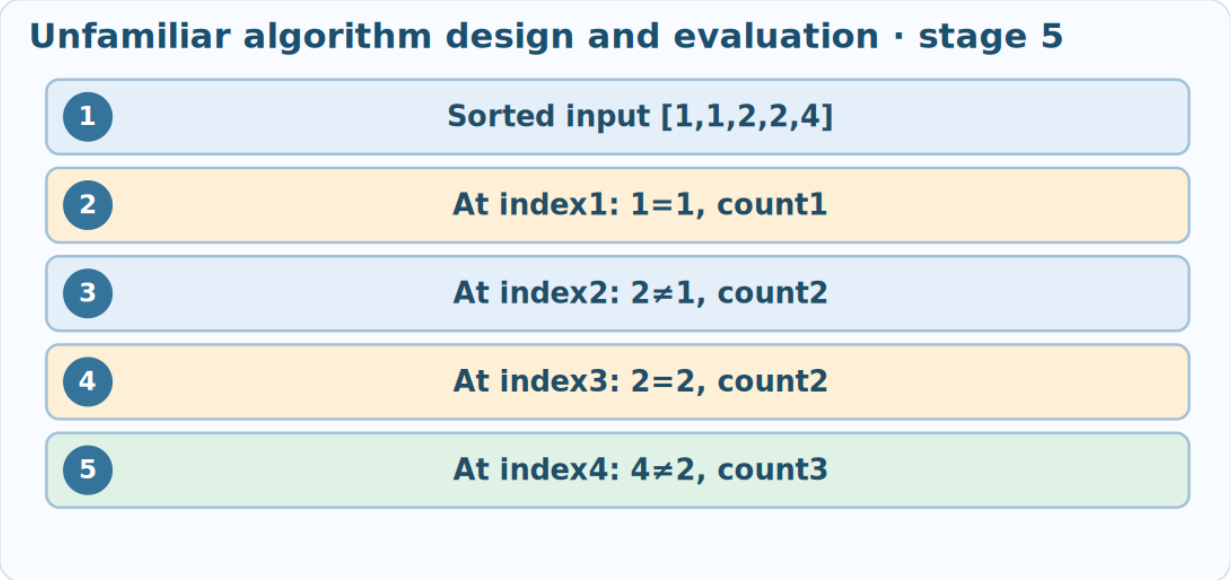


Unfamiliar algorithm design and evaluation · stage 4; Sorted input [1,1,2,2,4]; At index1: 1=1, count1; At index2: 2≠1, count2; At index3: 2=2, count2

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Unfamiliar algorithm design and evaluation · stage 5; Sorted input [1,1,2,2,4]; At index1: 1=1, count1; At index2: 2≠1, count2; At index3: 2=2, count2; At index4: 4≠2, count3

I DO — WORKED EXAMPLE

Follow the worked method

Trace distinct count in [2,2,5,5,7,7,7].

Predict first

Duplicates do not increment the counter because each equals the preceding element.

Explain the method

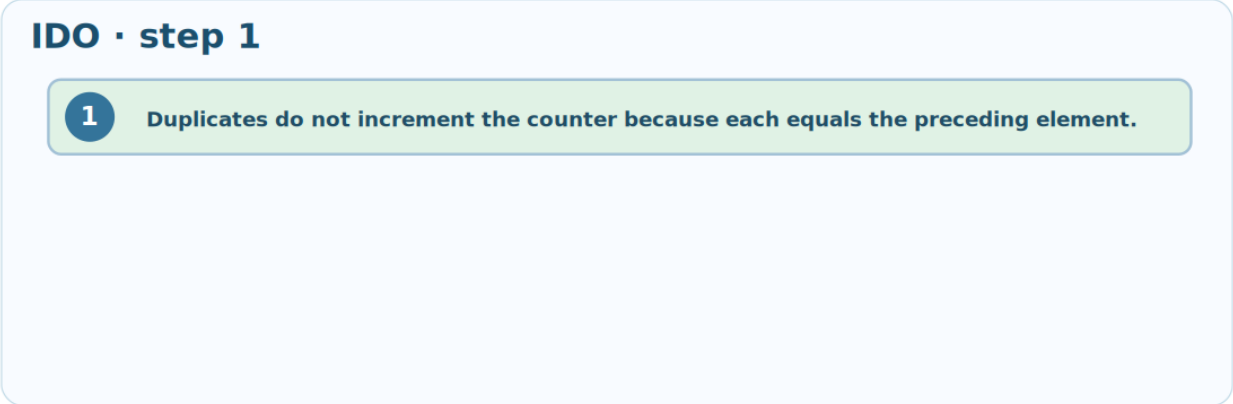
Only changes to five and seven increment count; result is three distinct values.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace distinct count in [2,2,5,5,7,7,7].

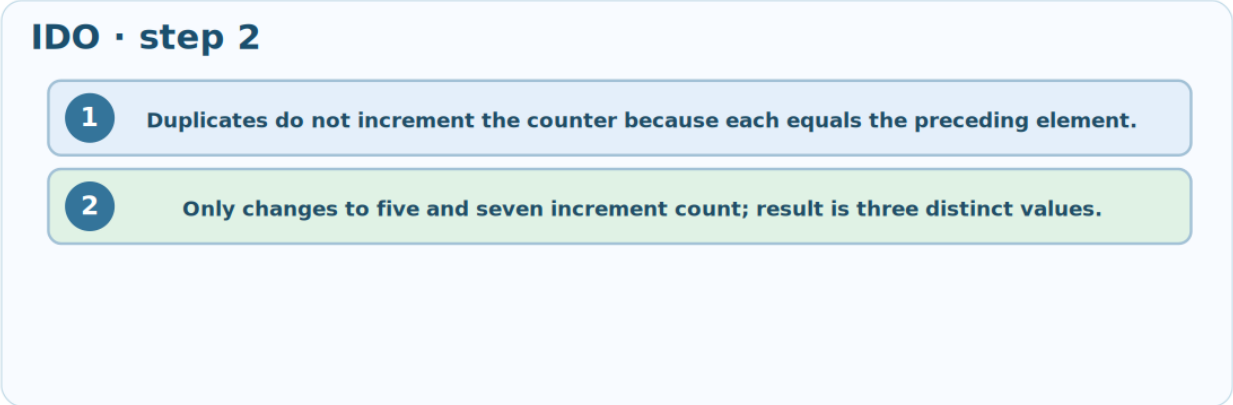
Duplicates do not increment the counter because each equals the preceding element.



IDO · step 1; Duplicates do not increment the counter because each equals the preceding element.

Trace distinct count in [2,2,5,5,7,7,7].

Duplicates do not increment the counter because each equals the preceding element.



IDO · step 2; Duplicates do not increment the counter because each equals the preceding element.; Only changes to five and seven increment count; result is three distinct values.

WE DO — GUIDED PRACTICE

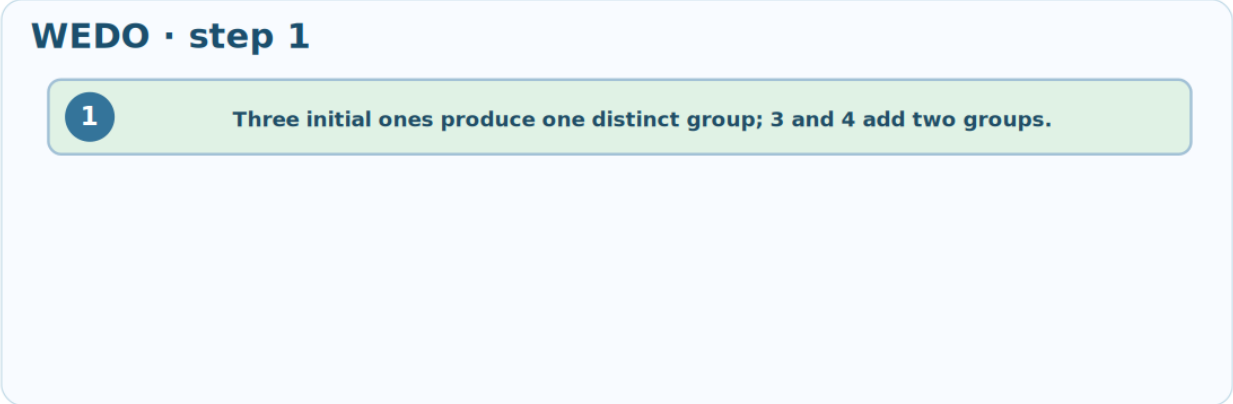
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace the algorithm on [1,1,1,3,4,4] and state two edge tests.

Three initial ones produce one distinct group; 3 and 4 add two groups.



WEDO · step 1; Three initial ones produce one distinct group; 3 and 4 add two groups.

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict first Three initial ones produce one distinct group; 3 and 4 add two groups.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Trace DistinctCount on [3,3,3,6,8,8]. Record count after each index. [6]

Question 2

Describe a correct set of test cases for the algorithm and their expected outputs. [6]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Explain why the algorithm may fail on unsorted [2,1,2]. Give a repair and complexity consequence. [6]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

The distinct-count algorithm is valid for unsorted inputs without modification.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

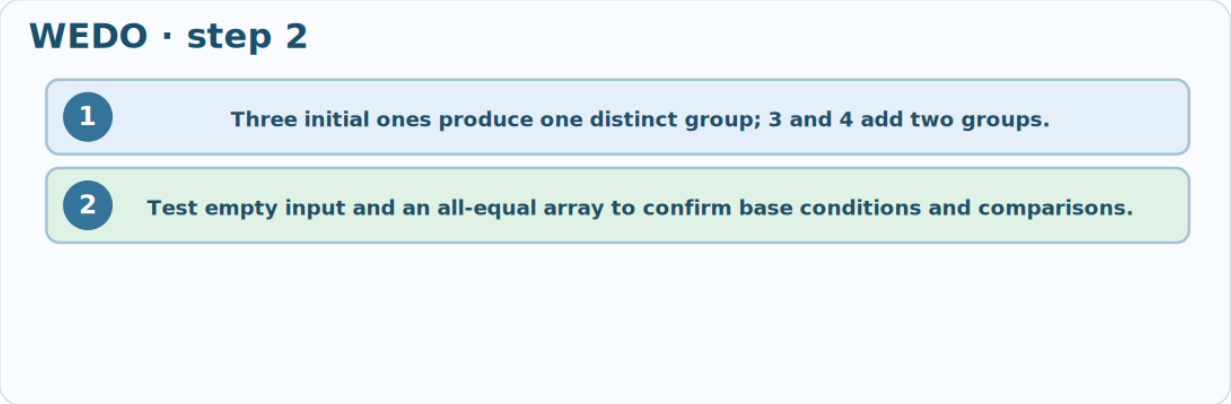
Next step: revisit the model or a relevant revision-guide section if needed.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace the algorithm on [1,1,1,3,4,4] and state two edge tests.

Three initial ones produce one distinct group; 3 and 4 add two groups.



WEDO · step 2; Three initial ones produce one distinct group; 3 and 4 add two groups.; Test empty input and an all-equal array to confirm base conditions and comparisons.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Explain the method

Test empty input and an all-equal array to confirm base conditions and comparisons.

IMPROVE: Explain one correction to your own reasoning.